

CVE-2026-46242 Deep Dive

Root on Linux, DoS on Android

Aayush Bankar

Cybersecurity Analyst @ CypherMatrix

Tier 1: github.com/Aayushbankar/bad-epoll-lab

Tier 2: github.com/Aayushbankar/bad_epoll_ab_tier2

Original vuln by **Jaeyoung Chung** via Google kernelCTF

August 2026

Independent reproduction & exploitability research

No 0-days disclosed

Contents

1	Scope & Attribution	4
2	Executive Summary	5
3	Introduction	6
4	CVE-2026-46242: The Basics	6
5	Environment & Reproducibility	7
5.1	Tier 1 — x86_64 (Fedora 44, Linux 6.12.67, QEMU)	7
5.2	Tier 2 — Android ARM64 GKI (QEMU aarch64 + GDB)	7
6	The Vulnerability: Race in the Heart of I/O	8
7	struct eventpoll: The Victim Object	10
8	Tier 1: Root on x86_64	12
8.1	Tier 1 Constants & Offsets	12
9	Tier 2: Android ARM64 Assessment	14
9.1	Slab Cache Journey	14
9.2	msg_msg Reclaim Mechanics	14
10	The 21 Dead Ends	16
10.0.1	Chain 0 — Controlled Crash via <code>percpu_counter_dec</code>	16
10.0.2	Chain 1 — KASLR Leak via Dual-Watch	16
10.0.3	Chain 2 — Arbitrary Decrement via Fake <code>user_struct</code>	17
10.0.4	Chain 3 — Full LPE	17
10.0.5	Additional Dead Ends	17
11	The Schedulability Wall	18
11.0.1	Natural Race Attempts	18
12	Why Android's Mitigations Worked	19
12.1	Control-Flow Integrity	19
12.2	Memory Safety	19
12.3	Scheduling	19
13	Lessons & Takeaways	20
14	Open Questions	21
15	Disclosure & Attribution	22
A	Verification Ledger — Summary (42 entries)	24
B	Dead Ends Register — 21 Entries	25
C	Experiment Index — NAT & EXP	25
D	Glossary	25

E	Repositories & Reproducibility	25
F	About the Author	26

1 Scope & Attribution

[TARGET] Original Discovery & Exploit

CVE-2026-46242 (“Bad Epoll”, CVSS 7.8) was discovered, analyzed, and exploited by **Jaeyoung Chung** (github.com/J-jaeyoung/bad-epoll) via Google’s kernelCTF. Patched upstream in [a6dc643c693](#) (v6.11 / v7.1-rc1).

[TOOL] This Project’s Contribution

1. **Tier 1 (x86_64/QEMU)**: Independent reproduction, GCC 16 toolchain port, database regeneration, runtime verification of the kernelCTF chain.
2. **Tier 2 (ARM64 Android 14 GKI)**: Original portability/exploitability research — did the primitive survive PAC, BTI, kCFI, MTE, slab isolation? **Documented negative result: DoS-only.**

[!] Tier 2 Test Methodology Disclosure

The Tier 2 assessment did **not** run against a stock/production Android kernel.

Stock Android GKI 6.1.23 (base commit 7e35917775b8) is NOT vulnerable to CVE-2026-46242 — the introducing commit [58c9b016e128](#) (v6.4-rc1) is absent from the 6.1 branch. This matches discoverer Jaeyoung Chung’s own affected-version statement (bug introduced in v6.4, so Pixel 8 et al. are safe).

To study what Android’s mitigation stack does to *this* primitive, we **deliberately cherry-picked the 6.1.y backport a1f93804449d into GKI 6.1.23** to build a vulnerable test kernel. Every Tier 2 result below (0/102,740 natural race hits, 21 dead ends, DoS-only) is a **portability/mitigation study of a synthetic backported-vulnerable kernel** — **not** a finding about any production Android device.

Correct framing: “If CVE-2026-46242 existed on GKI 6.1, would Android’s mitigations stop it?” — **not** “We tested a real vulnerable Android kernel.”

2 Executive Summary

Verdict: AMBER — DoS-only negative result, scientifically defensible (not a failure).

The UAF is **real and 100% reproducible with debugger assistance** (hardware watchpoint VER-026) and `msg_msg` reclaim of freed `struct eventpoll` in `kmalloc-192` is **reliable** (VER-027). However the race is **not naturally schedulable**: 0/102,740 unaided attempts across NAT-001 and NAT-005 (VER-034/039). All four exploitation chains are structurally dead (21 dead ends); the only remaining primitive is a fixed NULL write at offset 160, which the EXP-016 audit proves is DoS-only on this config. **Recommendation**: 2-week timebox on physical ARM64 hardware to test timing-widening; if 0 hits in $\sim 1\text{M}$ iterations, conclude DoS-only. Full evidence in `tier2/docs/VERIFICATION_LEDGER.md` (42 entries), `DEAD_ENDS_REGISTER.md` (21), `ASSUMPTIONS_REGISTER.md` (42).

3 Introduction

Every Linux process handling network connections, file descriptors, or event loops uses `epoll`. `nginx`, `Node.js`, Android's `SurfaceFlinger`, `Binder` — all depend on it. When a UAF lands here, blast radius = everything.

CVE-2026-46242 (Bad Epoll) is a race in the kernel's `epoll` implementation that frees a critical structure while another thread is still writing to it.

- **x86_64**: Ported & reproduced the kernelCTF chain → working root shell (`UID: 0`).
- **ARM64 Android GKI 6.1 (backported-vuln build)**: 21 dead ends, **0/102,740** natural race hits, verdict = **DoS only** (on synthetic build, not stock Android).

This is the full technical story: vuln mechanics, working `x86_64` chain, systematic ARM64 assessment, every failed chain & why, what Android's mitigations actually stop.

4 CVE-2026-46242: The Basics

Table 1: CVE-2026-46242 Summary

Field	Value
CVE	CVE-2026-46242 (CVSS 7.8)
Name	Bad Epoll
Class	Use-After-Free (race condition)
Subsystem	<code>fs/eventpoll.c</code>
Discoverer	Jaeyoung Chung (github.com/J-jaeyoung/bad-epoll) via kernelCTF
Introduced	v6.4-rc1 (58c9b016e128)
Patched	v6.11 / v7.1-rc1 (a6dc643c693)
Affected	Linux 6.4–6.10.x, Android GKI $\geq$ 6.4. GKI 6.1 NOT affected (commit 58c9b016e128 absent).
Impact	LPE to root (<code>x86_64</code> kernelCTF); DoS-only on backported-vuln GKI 6.1 test build.

Evidence-First Methodology

- **42** verification entries mapped to GDB hardware watchpoints / static audits
- **3** retracted claims (VER-010, 029, 030) — kept visible with reasons
- **21** dead ends with killing evidence
- **42** assumptions tracked (validated → falsified)
- All raw evidence (GDB logs, serial output, disassembly) in repos

Full chain: `VERIFICATION_LEDGER.md`, `DEAD_ENDS_REGISTER.md`, `ASSUMPTIONS_REGISTER.md`

5 Environment & Reproducibility

5.1 Tier 1 — x86_64 (Fedora 44, Linux 6.12.67, QEMU)

- Host: Fedora 44, x86_64, GCC 16.1.1, QEMU
- Kernel: 6.12.67 compiled with DWARF BTF, nokaslr (KASLR disabled for determinism), base 0xffffffff81000000
- Target DB: lts-6.12.67 rebuilt via rp++ + angrop (patched PicklingError in SpecialMem)
- Offsets (GDB-mapped, vs Google defaults): task_struct.comm 1840 (was 1928), files 1896 (1984), children 1384 (1472), sibling 1400 (1488)
- Verified gadgets: POP_RDI_RET 0x1600ea0 (0xffffffff81600ea0: pop rdi; jmp __x86_return_thunk), POP_RET 0x42010, __x86_return_thunk 0x20d6e70
- Race tuning: RACE_DUP_CLOSE_ITERS 20 (was 250), RACE_AHEAD_HI 10000ns (4000ns), timeout 600s
- Rootfs: custom initramfs.cpio with busybox, /init mounts /proc, /sys, payload at exploit/tier1/rootfs_debug/bin/exploit
- Evidence: qemu_output.log (Win! UID 0), target_db.kxdb SHA256 63542e7f..., exploit SHA256 95cd9f36...

Rebuild: make kernel → start_qemu.sh → tail -f qemu_output.log. Gadget DB must match exact vmlinux; see knowledge/02_projects/bad-epoll/01_tier1/02_env_rebuild_guide.md and 03_offsets_layout.md.

5.2 Tier 2 — Android ARM64 GKI (QEMU aarch64 + GDB)

- Base: Android 14 GKI 6.1.23 @ 7e35917775b8, stock **NOT vulnerable** (no 58c9b016e128); test kernel = base + cherry-picked 6.1.y backport a1f93804449d (deliberately backported-vulnerable)
- Artifacts: tier2/android/artifacts/Image, vmlinux (285 MB, debug_info, 98575 symbols via System.map), initramfs.cpio (BusyBox)
- Toolchain: aarch64-linux-gnu-gcc (static), qemu-system-aarch64 -nographic -s -S, gdb-multiarch, gdb_race_test.py (sync_flag + dmb sy; b . patch)
- Config: PREEMPT_VOLUNTARY, CONFIG_PREEMPT_DYNAMIC=y (default voluntary), SLAB_ACCOUNT on, CONFIG_MEMCG=n, CONFIG_UNUSED_KSYMS_WHITELIST patched
- Reclaim: msg_msg 144B payload in kmalloc-192, marker 0xdead000000000000 at user byte 88 (offset 136) verified via GDB dump

Runbook: tier2/docs/TIER2_MANUAL_RUNBOOK.md (redacted, now at tier2/docs/TIER2_MANUAL_RUNBOOK.md); 2-min demo test_gdb.sh + gdb -x gdb_race_test.py vmlinux.

6 The Vulnerability: Race in the Heart of I/O

The bug lives in `__ep_remove()` vs `eventpoll_release()` — concurrent execution when nested `epoll` fds are closed from different threads.

Setup: Two `epoll` instances — outer watches inner.

```
1 int outer = epoll_create1(0);
2 int inner = epoll_create1(0);
3 epoll_ctl(outer, EPOLL_CTL_ADD, inner, &ev); // inner watched by outer
```

Listing 1: Nested `epoll` setup

Thread A (`close(outer)`) → `__ep_remove()` clears backpointer:

```
1 WRITE_ONCE(file->f_ep, NULL); // Clears inner file's epoll backpointer
```

Listing 2: Thread A clears backpointer

Thread B (`close(inner)`) → `__fput()` → `eventpoll_release()` checks locklessly:

```
1 if (!file_has_epoll(f)) // Reads f_ep --- sees NULL from Thread A
2     return;              // Skips epoll cleanup entirely
```

Listing 3: Thread B lockless fast path

Thread B sees `NULL`, assumes no `epoll` refs, **frees the struct `eventpoll`** (inner fd). Memory returns to `kmalloc-192`.

Thread A continues `__ep_remove()`:

```
1 hlist_del_rcu(&epi->flink); // Writes to inner_epoll->refs at offset 160
```

Listing 4: Thread A UAF write

8-byte `NULL` write at offset 0xA0 (160) of already-freed struct `eventpoll`. This is the UAF.

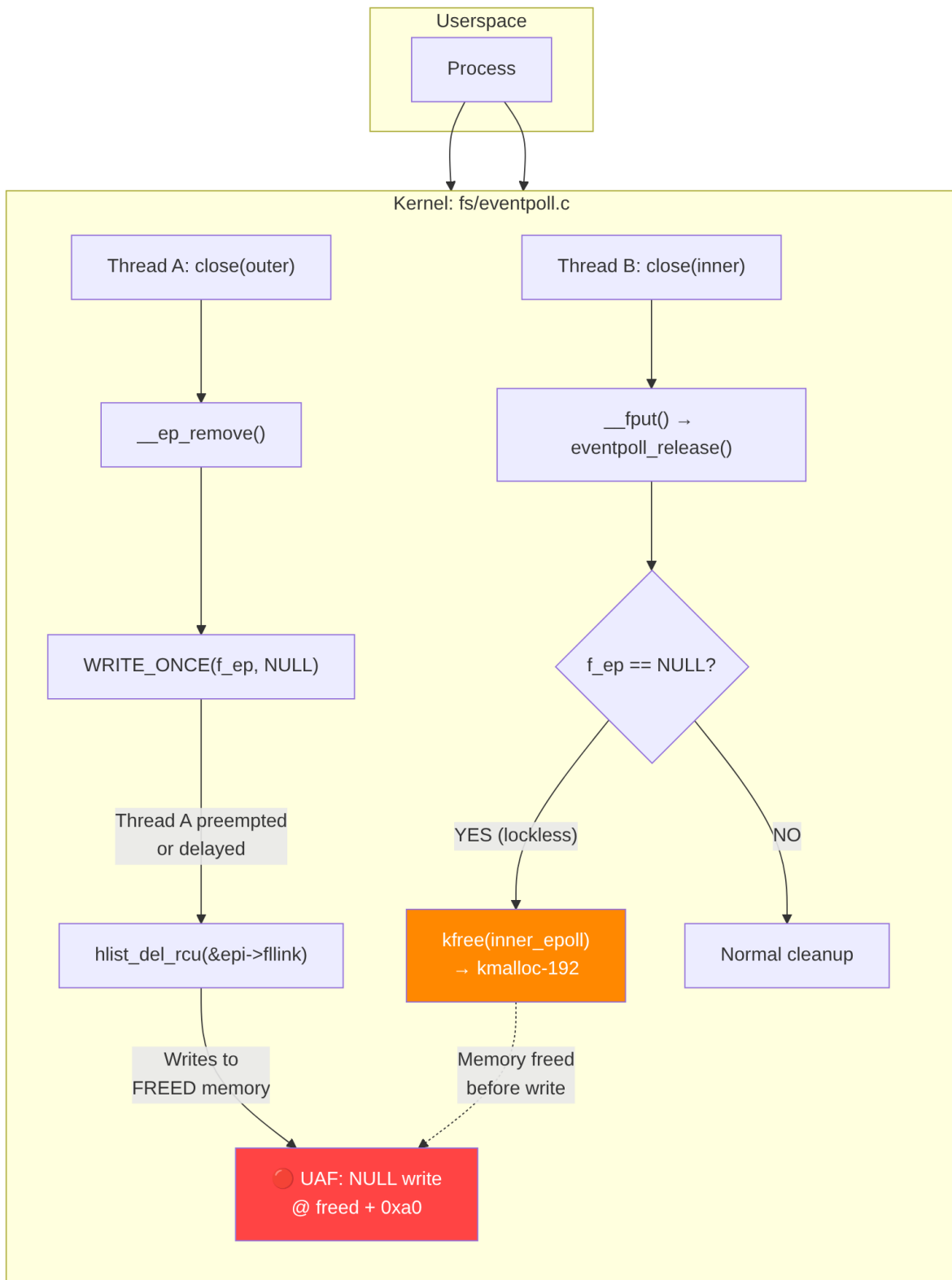


Figure 1: Attack Surface — Userspace to Kernel UAF

7 struct eventpoll: The Victim Object

struct eventpoll = 176 bytes → kmalloc-192. Field at offset 160 (refs, struct hlist_head) is the UAF target:

struct eventpoll (176 bytes → kmalloc-192)

0x00:	wait_queue_head_t wq	(24 B)	
0x18:	wait_queue_head_t poll_wait	(24 B)	
0x30:	struct list_head rdllist	(16 B)	
0x40:	struct rb_root_cached rbr	(16 B)	
0x50:	struct epitem *ovflist	(8 B)	
0x58:	struct wakeup_source *ws	(8 B)	
0x60:	struct user_struct *user	(8 B)	
0x68:	struct file *file	(8 B)	
0x70:	... (misc)	(48 B)	
0xA0:	struct hlist_head refs ← UAF WRITE		← offset 160
0xA8:	... (remaining)	(8 B)	
0xB0:	end	(176 B)	

When freed to kmalloc-192, attacker sprays msg_msg (SysV msgsnd 144B user data). msg_msg header = 0x00-0x2F (48B), attacker payload = 0x30-0xBF. UAF NULL write at 0xA0 lands **inside attacker payload** — zeroes 8 bytes of their own message, no kernel pointer corruption.

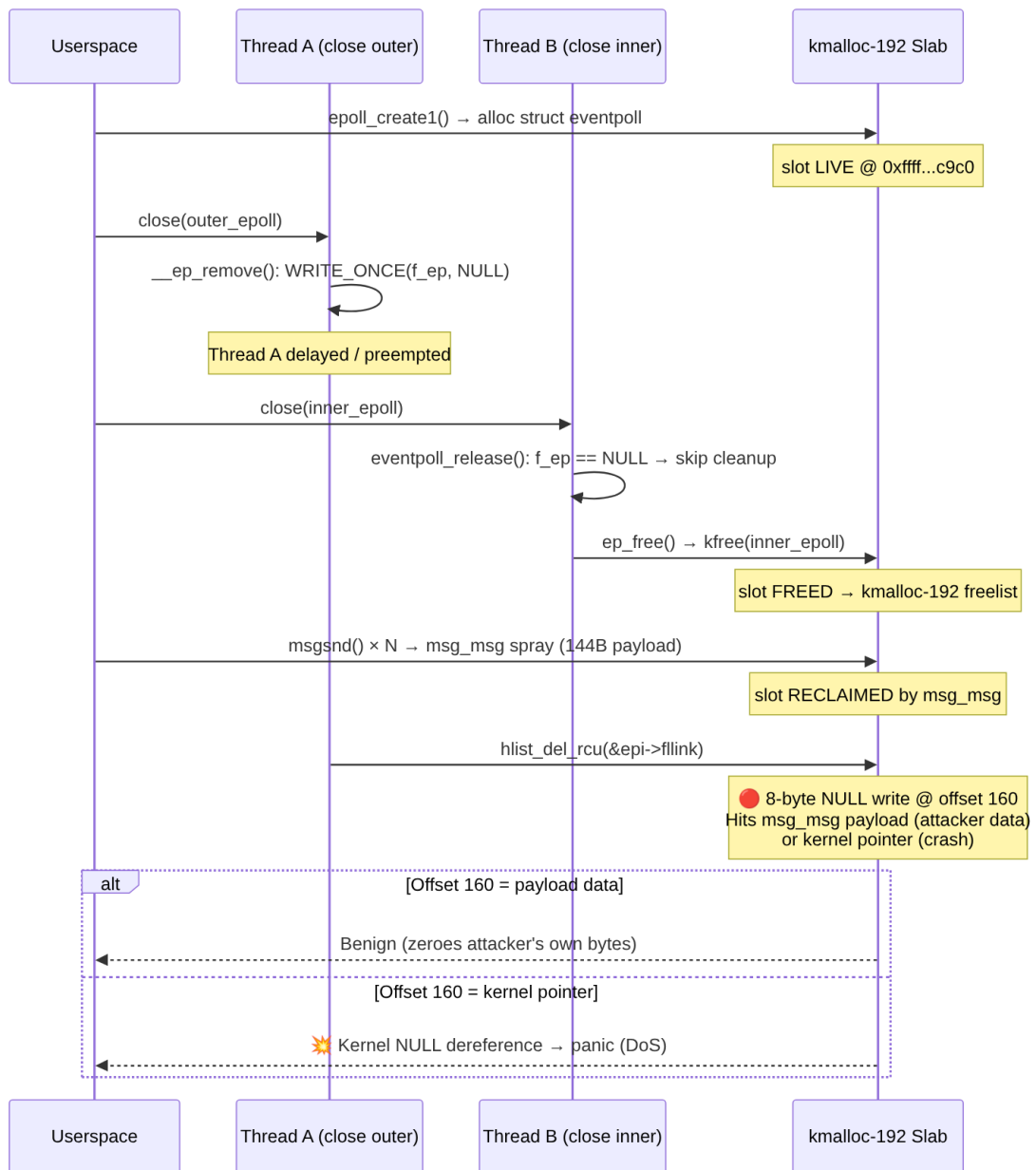


Figure 2: Race Timeline — Alloc → Free → Reclaim → UAF

8 Tier 1: Root on x86_64

Env: Fedora 44, Linux 6.12.67 (GCC 16.1.1), QEMU + SMEP/SMAP/KPTI.

Jaeyoung Chung's kernelCTF exploit targets `struct epitem` (120B, `eventpoll_epi` cache on newer kernels, cross-cacheable on COS target) + `struct file`, not `struct eventpoll`.

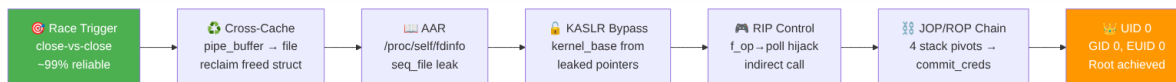


Figure 3: Tier 1 Exploitation Chain

Stage 1 — Race: Two threads race `close()` on nested `epoll` FDs. ~99% success after tuning.

Stage 2 — Cross-cache: Freed `struct file` reclaimed via `pipe_buffer` spray → attacker controls `f_op / f_inode`.

Stage 3 — AAR: Forge `f_inode` → `/proc/self/fdinfo` (`seq_file`) → leak `task_struct` → KASLR base.

Stage 4 — RIP control: Overwrite `file->f_op->poll` → `poll()` triggers hijack.

Stage 5 — JOP/ROP: 4 stack pivots via `__x86_indirect_call_thunk_rdi` → RSP onto `mmap'd` page → `commit_creds(&init_cred)` → `SWAPGS` return → `UID 0`.

```
cross-cache ok
kernel_base=ffffffff81000000
task=ffff8880...
READY_FOR_GDB
Win! UID: 0, GID: 0, EUID: 0
```

[IDEA] The Offset Desync Breakthrough Initial crash: Supervisor Write Fault at `0xffffffff810001bd` (dead padding). Root cause: upstream `target_db.kxdb` built for Google COS kernel. GCC 16 vs Clang 12 shifts every gadget offset. Fix: regenerate DB with `rp++ + angrop` on local `vmLinux` — required patching `PicklingError` in `angrop` (nested `SpecialMem` class couldn't pickle).

Remaining: Userland return leaves RSP misaligned → `execve("/bin/sh")` `SIGSEGV`. `PID 1 / init` panics kernel. Escalation proven (`UID 0`), shell unstable. Post-exploitation issue, not primitive failure.

8.1 Tier 1 Constants & Offsets

Full reference at `knowledge/02_projects/bad-epoll/01_tier1/03_offsets_layout.md`:

Table 2: `task_struct` Offsets (GDB-mapped vs Google defaults)

Field	Google	Our Build	Purpose
<code>comm</code>	1928	1840	AAR verification
<code>files</code>	1984	1896	fd table
<code>children</code>	1472	1384	process tree walk
<code>sibling</code>	1488	1400	process tree walk

Table 3: Verified Gadgets (6.12.67, GCC 16.1.1, RETPOLINE=y)

Define	Offset	Addr	Insn
POP_RDI_RET	0x1600ea0	0xffffffff81600ea0	pop rdi; jmp __x86_return_thunk
POP_RET	0x42010	0xffffffff81042010	ret (c3)
__x86_return_thunk	—	0xffffffff820d6e70	ret

CONFIG_RETPOLINE=y → all ret → jmp __x86_return_thunk; PIVOT1 candidates mov rsp,rdi/rdx, xchg rsp,* none found in 8.5M lines objdump; failed PIVOT1 0x10d6525 = store, not pivot.

9 Tier 2: Android ARM64 Assessment

Env: Android 14 GKI, kernel 6.1.23 (base 7e35917775b8), QEMU aarch64 + GDB. **Stock GKI 6.1.23 NOT vulnerable** (commit 58c9b016e128 absent). Test kernel = base + cherry-picked 6.1.y backport a1f93804449d = **deliberately backported-vulnerable build**.

Goal: *Hypothetical* — **if** Bad Epoll existed on GKI 6.1, could it escalate? Portability/mitigation study, **not** a claim about production devices.

9.1 Slab Cache Journey

Initial assumption: struct epitem (120B) in kmalloc-128 or mergeable. **Wrong**.

- struct epitem in **dedicated** eventpoll_epi cache (SLAB_HWCACHE_ALIGN | SLAB_PANIC | SLAB_ACCOUNT at eventpoll.c:2555)
- SLAB_ACCOUNT prevents merge with kmalloc-128 (verified slab_common.c:173-215)
- CONFIG_MEMCG=n → no kmalloc-cg-128
- Cross-cache reclaim from generic kmalloc = **impossible**

Killed pipe_buffer → struct file cross-cache (worked on x86_64). Killed snd_timer_user + all kmalloc-192 sprays for epitem.

Breakthrough: Real victim = struct eventpoll (176B, generic kmalloc-192). Race puts freed eventpoll on kmalloc-192 freelist → msg_msg reclaims it.

9.2 msg_msg Reclaim Mechanics

msgsnd() 144B user data → msg_msg in kmalloc-192:

msg_msg in kmalloc-192 (reclaiming freed eventpoll)

0x00-0x2F: msg_msg header (kernel-controlled)
0x30-0xBF: user payload (144 bytes, attacker-owned)
0xA0: ##### UAF NULL WRITE HERE #####
0xC0: end (192 byte slot)

← inside payload

GDB verified: marker 0xdead000000000000 at user byte 88 (slab offset 136) appeared correctly. Reclaim reliable **under GDB assistance** (infinite-loop patch creates 2-3s spray window, doesn't exist naturally).

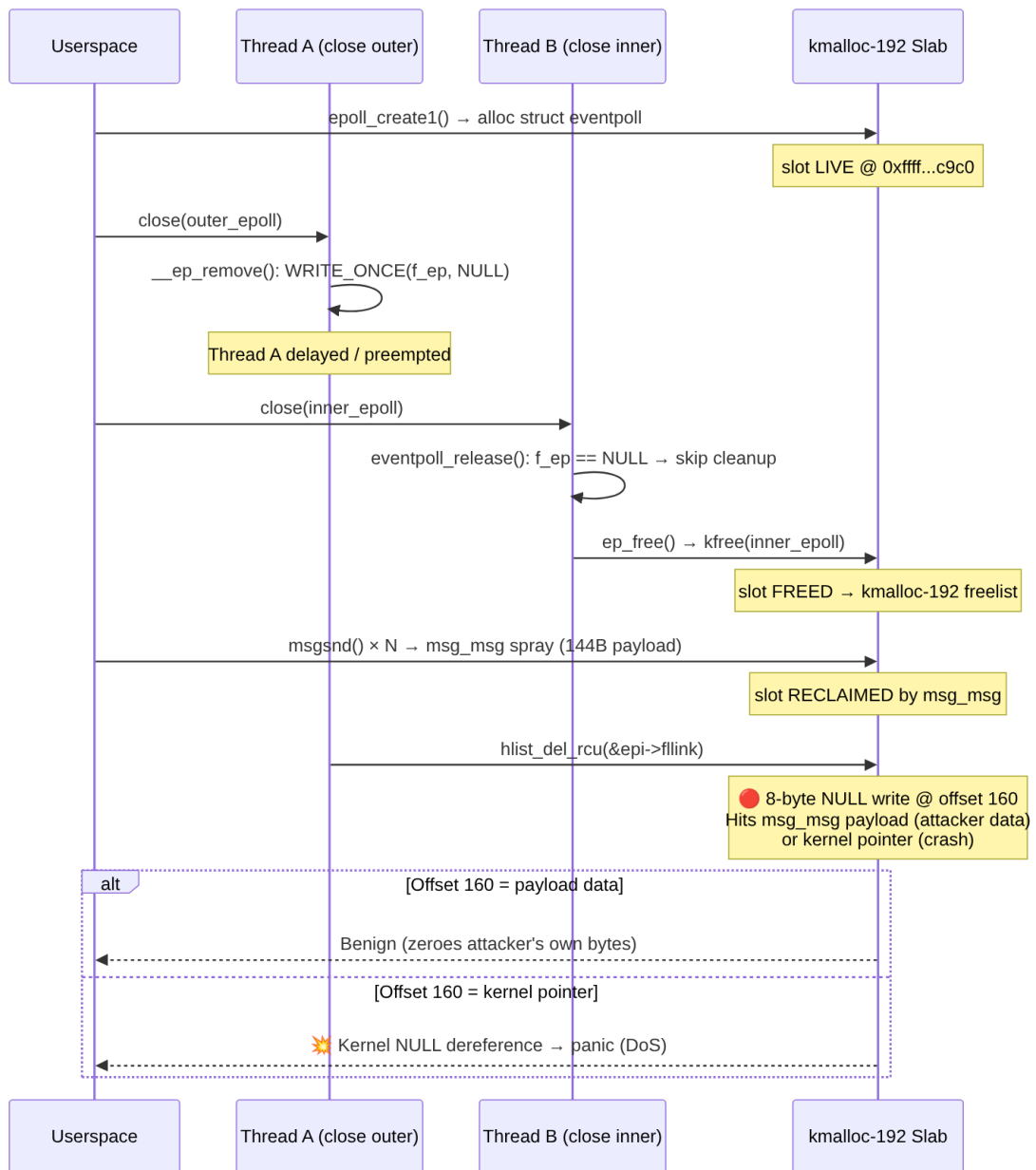


Figure 4: msg_msg Reclaim Mechanics

10 The 21 Dead Ends

Four chains hypothesized, investigated, disproven — each with killing evidence.

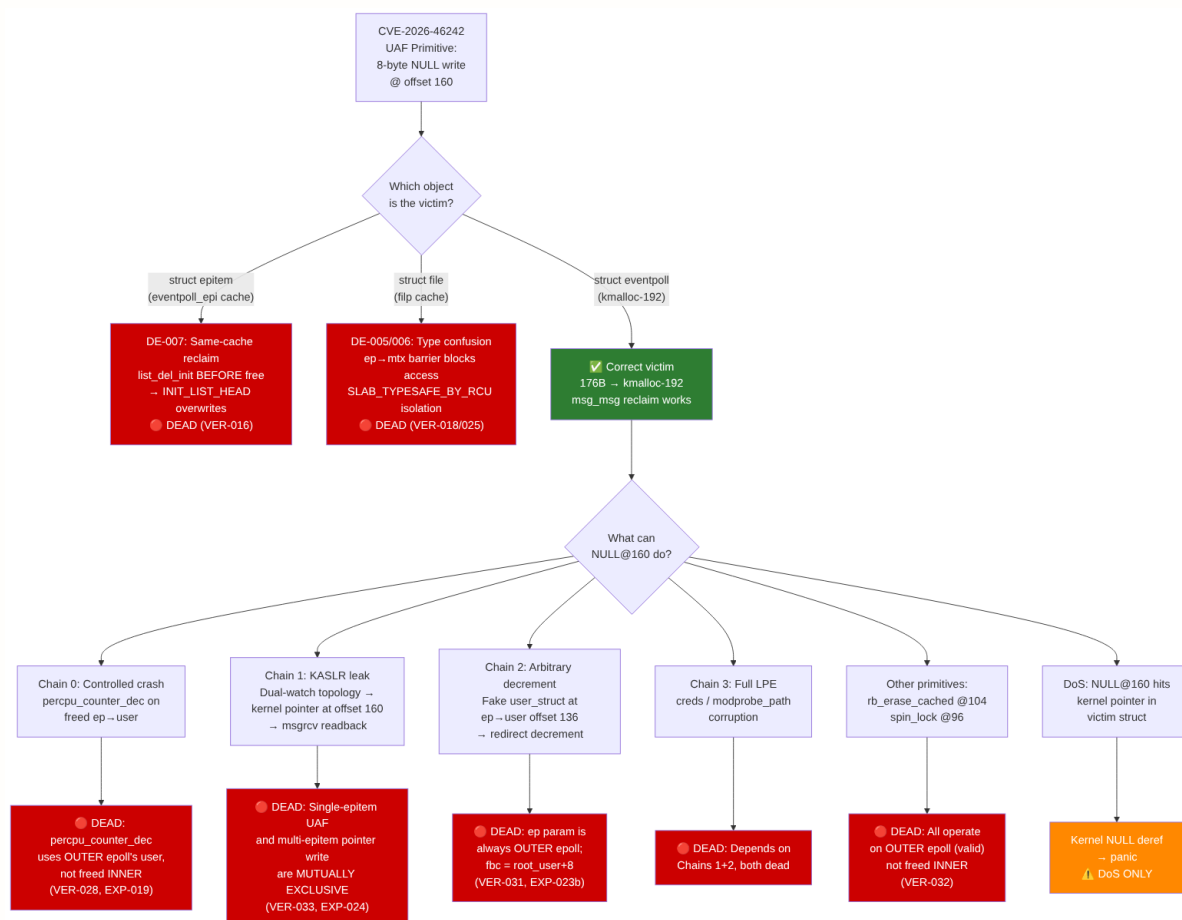


Figure 5: Exploitation Decision Tree

10.0.1 Chain 0 — Controlled Crash via `percpu_counter_dec`

Hypothesis: Reclaim freed eventpoll with `msg_msg` containing fake `user_struct` at offset 136 → `__ep_remove()` calls `percpu_counter_dec(&ep->user->epoll_watches)` → deref attacker pointer.

Killing (VER-028, EXP-019): `percpu_counter_dec` uses **outer** eventpoll's user (valid), not freed inner. `ep` param always outer. Marker at offset 136 placed but never read.

10.0.2 Chain 1 — KASLR Leak via Dual-Watch

Hypothesis: Two outer epolls watching inner → two epitemes. Race fires → `hlist_del_rcu` writes `epi2` kernel address (not NULL) at offset 160 → read via `msgrcv` → KASLR defeated.

Killing (VER-033, EXP-024): UAF condition (single epitem: `f_ep=NULL` path) and pointer-write condition (multi-epitem: `hlist_del_rcu` writes next node) are **mutually exclusive**. 2+ epitemes → `WRITE_ONCE(f_ep, NULL)` never executes → lockless bypass never triggers → inner never freed → `hlist_del_rcu` writes to **live** memory. HW watchpoint: `ep_free_inner_seen=False` at write.

10.0.3 Chain 2 — Arbitrary Decrement via Fake `user_struct`

Hypothesis: Fake `user_struct` at known addr → overwrite `ep->user` (offset 136) via spray → `percpu_counter_dec` decs attacker's counter (`modprobe_path / creds`).

Killing (VER-031, EXP-023b): Same as Chain 0. `percpu_counter_add_batch` showed `fbc = root_user+8` (outer `epoll`'s user). `ep` never freed object.

10.0.4 Chain 3 — Full LPE

Hypothesis: Chain 1 + Chain 2 → `creds / modprobe_path` corruption.

Killing: Both prerequisites dead.

10.0.5 Additional Dead Ends

Table 4: Additional Dead Ends

Path	Why Dead
<code>rb_erase_cached</code> arb write @104	Operates on outer <code>epoll</code> 's <code>rbr</code> (valid)
<code>spin_lock_irq</code> corrupt @96	Operates on outer <code>epoll</code> 's <code>lock</code> (valid)
<code>struct file</code> type confusion	<code>ep->mtx</code> blocks concurrent access to stale <code>epitem</code>
<code>pipe_buffer</code> → <code>struct file</code> cross-cache	<code>SLAB_TYPESAFE_BY_RCU</code> + size mismatch
<code>epitem</code> same-cache reclaim	<code>list_del_init</code> before <code>call_rcu</code> — corrupt doesn't survive
<code>snd_timer_user</code> reclaim	Different slab cache entirely
<code>eventpoll_epi</code> → <code>kmalloc-128</code> merge	<code>SLAB_ACCOUNT</code> prevents merge

11 The Schedulability Wall

Even with a useful primitive, race never fires without GDB.

11.0.1 Natural Race Attempts

Table 5: Natural Race Attempts

Experiment	Iterations	Hits	Optimizations
NAT-001	10,000	0	Baseline, 2 CPUs
NAT-005	92,740	0	isolcpus=1, 4MB cache eviction, 10-cycle delay steps, closed-loop calibration
Total	102,740	0	Best alignment: 1 cycle (~16 ns) at delay=2,360

- Root cause:
- cond_resched() at eventpoll.c:888/903 = **no-op**: dynamic_cond_resched key = FALSE under CONFIG_PREEMPT_DYNAMIC default PREEMPT_VOLUNTARY; TIF_NEED_RESCHED never set on pinned CPU (VER-035)
 - __ep_remove = **zero** preemption points (disassembly)
 - Vulnerable sequence runs atomically wrt scheduling
 - Real window: ~250–550 cycles (~125–275 ns @ 2 GHz) — below scheduling granularity

Natural Race Hit Rate (QEMU TCG)

NAT-001: 0 / 10,000 (CI upper bound: 0.0384%) [————] 0%

NAT-005: 0 / 92,740 (extreme optimization) [————] 0%

Combined: 0 / 102,740 [————] 0%

95% CI upper bound: < 0.003%

Required for exploit: ≥ 0.01% (conservative)

Race window: WRITE_ONCE(f_ep,NULL) —250-550 cycles —>UAF

Thread A writes Thread B must see NULL, free, and Thread A

then continues must resume in this window

Scheduling granularity: ~4,000,000 cycles

Race window: ~400 cycles

Ratio: 1 : 10,000

Best alignment: 1 cycle error (~16 ns) Still: 0 hits

Figure 6: Why 0/102,740 Is Conclusive

[HW] Hardware Caveat: QEMU TCG doesn't model real cache-coherency latency, store buffer propagation, or asymmetric core clock drift. On physical ARM64 (Cortex-X4 3.2 GHz + Cortex-A520 1.8 GHz), hardware effects create natural phase shifts that *could* widen the window. First-principles analysis says race is **physically viable** on real silicon — but primitive stays capped at DoS regardless.

12 Why Android's Mitigations Worked

Even with 100% reliable race on hardware, mitigation stack blocks escalation:

12.1 Control-Flow Integrity

Table 6: Control-Flow Integrity

Mitigation	Blocks
PAC	Function ptrs cryptographically signed. Forging <code>f_op->poll</code> → HW fault. PACMAN (MIT CSAIL '22) shows speculative oracles in userland, but unguided NULL-write lacks arbitrary read oracle for valid signatures.
kCFI	Clang validates 32-bit type hashes before indirect <code>blr</code> . Valid addr fails if hash mismatches.
BTI	Indirect branches must land on <code>btb</code> landing pads. Random gadgets unreachable.

12.2 Memory Safety

Table 7: Memory Safety

Mitigation	Blocks
MTE	4-bit tags / 16-byte granule. Freed memory access with mismatched tag → sync HW fault. TikTag ('24) shows speculative tag leakage, but async multicore noise prevents reliable recovery in unguided race → stale UAF = immediate HW exception.
SLAB_TYPESAFE_BY_RCU	<code>filp_cache</code> can't be reclaimed by different caches.
SLAB_ACCOUNT	<code>eventpoll_epi</code> fully isolated from generic <code>kmalloc</code> .

12.3 Scheduling

Table 8: Scheduling

Mitigation	Blocks
PREEMPT_VOLUNTARY	<code>__ep_remove</code> = 0 preemption points. Vulnerable seq runs atomically.
No cond_resched	<code>dynamic_cond_resched</code> = FALSE; <code>TIF_NEED_RESCHED</code> never set on pinned CPUs.

x86_64 bypassed KASLR/SMEP/SMAP/KPTI via AAR/JOP/ROP/SWAPGS. ARM64 adds PAC/BTI/kCFI/MTE/slab isolation — and they held.

13 Lessons & Takeaways

1. **Vuln $\neq$ Exploit.** CVE-2026-46242 real, verified, mechanically reproducible. x86_64 = root. Backported GKI 6.1 = kernel panic. Gap between “bug exists” and “bug exploitable” = the whole story. Stock GKI 6.1 never exposed (bug intro v6.4).
2. **Mitigation stack works in depth.** No single mitigation killed it — all contributed. PAC blocks ptr forgery, kCFI blocks indirect call hijack, SLAB blocks cross-cache, MTE detects stale access, PREEMPT_VOLUNTARY closes scheduling window. Each layer removes options until nothing viable.
3. **Negative results are signal.** 21 dead ends, 3 retractions, 0/102,740 hits = definitive: this is what vuln looks like when mitigations actually work. Too much research only publishes wins. Failures = signal.
4. **Evidence discipline prevents bias.** Phase 3 adversarial review (EVO-009) found **every** prior race demo was GDB-assisted — artificial preemption point absent in real kernel. Without ledger tracking *how* claims were proven, this goes unnoticed. Track methodology, not just results.
5. **Compiler changes break exploit portability.** Tier 1 failed initially: GCC 16 shifted every gadget vs Google’s Clang. Same kernel version, same source, different binary layout. `target_db.kxdb` must match exact executing binary.
6. **Attack surface shifting to drivers.** Core subsystems well-mitigated → remaining surface = vendor drivers (GPU, camera, audio) often lacking same hardening. Vendor GPU drivers = deterministic page-level UAFs + direct physmem manipulation → sidestep PAC/BTI/kCFI without ptr forgery.

14 Open Questions

DoS-only verdict for backported-vuln GKI 6.1.23 (stock GKI 6.1 unaffected). Open:

1. **Physical ARM64 validation:** Race fire naturally on real silicon (Cortex-X4 + Cortex-A520)? First-principles says yes, but rate? 2-week HW timebox ~1M iterations would settle.
2. **Alt kmalloc-192 victims:** EXP-016 covered `fib6_info`, `snd_timer_user`, `packet_fanout`, `urb`, `wakeup_source`. Other structs where NULL @160 corrupts something useful? Deep slab cross-ref on vuln GKI 6.1 builds?
3. **Different kernel configs:** Assessment on `PREEMPT_VOLUNTARY + PREEMPT_DYNAMIC=voluntary`. `PREEMPT_FULL` or vendor kernels add schedulable points in `__ep_remove`?
4. **Data-only attacks:** Control-flow hijacking blocked by PAC/BTI/kCFI. Data-only path from fixed NULL write (8B, 0x0, offset 160, single write)? Constraint brutal, but data-only attacks surprise us.

[CHAT] Collaborate? Vuln is public, repos linked, evidence chain documented. Fresh eyes on `kmalloc-192` or scheduling could flip Tier 2 verdict. Ping me.

15 Disclosure & Attribution

- **Original Vuln & Exploit:** Jaeyoung Chung, kernelCTF, github.com/J-jaeyoung/bad-epoll
- **Upstream Patch:** a6dc643c693 (v6.11 / v7.1-rc1)
- **This Research:** Independent Tier 1 x86_64 reproduction + original Tier 2 portability/mitigation study on deliberately backported-vuln Android 14 ARM64 GKI 6.1.23. Stock GKI 6.1 **NOT** affected (58c9b016e128 absent); test kernel = base + cherry-pick a1f93804449d. No 0-days.

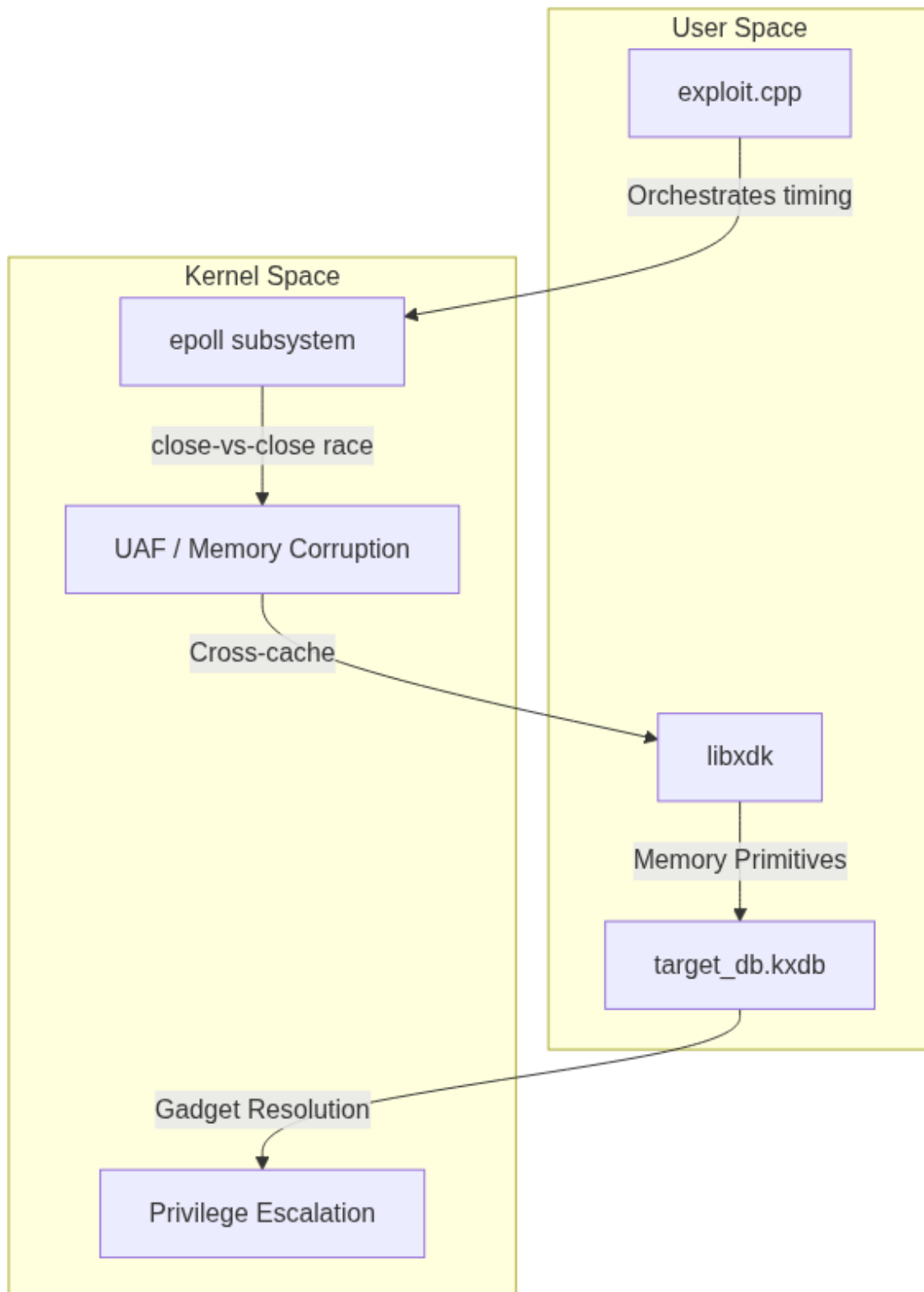


Figure 7: Architecture Map — Userland to Kernel Escalation Flow

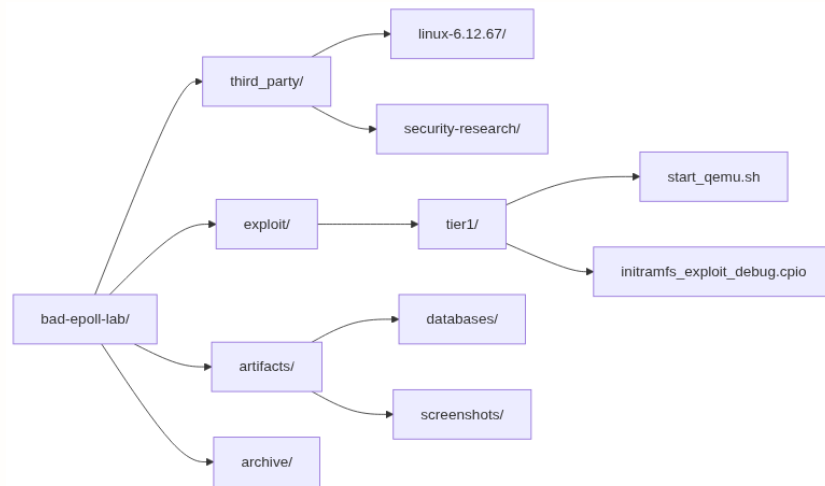


Figure 8: Repository Layout — Tier-1 (left) vs Tier-2 (right) separation

A Verification Ledger — Summary (42 entries)

Full ledger at `tier2/docs/VERIFICATION_LEDGER.md` and `knowledge/02_projects/bad-epoll/02_tier2/01_`.
Excerpt (VER-026/027/028/031/033/034/039 are load-bearing):

Table 9: Verification Ledger Excerpt

ID	Status	Claim
VER-026	VERIFIED	Race proven via HW watchpoint: Thread A <code>WRITE_ONCE(f_ep, NULL)</code> , Thread B lockless bypass, Thread A <code>hlist_del_rcu NULL@160</code> on freed object.
VER-027	VERIFIED	<code>msg_msg 144B</code> reclaims freed eventpoll (marker <code>0xdead...</code> @ offset 136).
VER-028	VERIFIED	<code>percpu_counter_dec</code> uses OUTER epoll's user, not freed inner — Chain 0 dead.
VER-031	VERIFIED	<code>fbcr=root_user+8</code> — arbitrary decrement dead.
VER-033	VERIFIED	Single-epitem UAF vs multi-epitem pointer-write mutually exclusive — Chain 1 dead (retracts VER-029/030).
VER-034/039	VERIFIED	NAT-001 (10k) + NAT-005 (92,740) = 0/102,740 natural hits.
VER-035	VERIFIED	<code>cond_resched no-op</code> , <code>__ep_remove 0</code> pre-emption points.

3 retracted (VER-010/029/030) kept visible with reasons.

B Dead Ends Register — 21 Entries

Full register at [tier2/docs/DEAD_ENDS_REGISTER.md](#) (also [knowledge/02_projects/bad-epoll/02_tier2/02_evidence/02_dead_ends_register](#)). All 21 mapped to killing VER/EXP; 4 chains + 7 additional paths listed in §??.

C Experiment Index — NAT & EXP

[tier2/docs/EXPERIMENT_INDEX.md](#) ([knowledge/02_projects/bad-epoll/02_tier2/01_evidence/01_experiment_index](#)):

ID	Iters	Outcome
NAT-001	10,000	0 hits, baseline 2 CPUs
NAT-005	92,740	0 hits, isolcpus+eviction+sweeper, best align 1 cyc
EXP-015/018/019	—	GDB race + msg_msg reclaim, Chain 0 killing
EXP-023b/024	—	Chain 1/2 mutual exclusion

D Glossary

Term	Meaning
UAF	Use-After-Free — free then use via dangling pointer
SLAB	Kernel allocator; <code>kmalloc-192</code> , <code>eventpoll_epi</code> dedicated, <code>SLAB_ACCOUNT/TYPESAFE_BY_RCU</code> isolation
KASLR	Kernel ASLR — base randomization, bypass via AAR leak
PAC/BTI/kCFI	ARM64 CFI: pointer signing, landing pads, type hashes
MTE	Memory Tagging — 4-bit tags/16B granule, stale access = fault
PREEMPT_VOLUNTARY	Voluntary preemption — <code>__ep_remove</code> has 0 points

E Repositories & Reproducibility

Repo	URL / Commit
Tier-1	github.com/Aayushbankar/bad-epoll-lab@publish/clean-and-writeup-2026-08-29
Tier-2	github.com/Aayushbankar/bad_epoll_ab_tier2@main
Base GKI	7e35917775b8 (stock NOT vuln) + backport a1f93804449d (vuln test)
Artifacts	tier2/android/artifacts/Image,vmlinux,System.map,initramfs.cpio
Evidence	tier2/evidence/2026-07-* (hashes, qemu logs, GDB traces)

Rebuild: see §5. No Mali hunt content included — out of scope for this Bad Epoll dossier (see separate [cve-primitive-hunt](#) KB).

F About the Author

Aayush Bankar — Cybersecurity Analyst @ CypherMatrix.

Research workflow augmented by AI-assisted agentic coding tools (local LLM orchestration) for code comprehension, hypothesis generation, evidence organization — all engineering decisions & runtime evidence manually verified. AI workflow documented in Tier 1 Final Writeup.

Connect: [GitHub] github.com/Aayushbankar · [LinkedIn] linkedin.com/in/aayushbankar

Thanks for reading. If you work in kernel exploitation, Android security, or mitigation engineering — perspective appreciated, especially on open questions. Best outcomes come from collaborative scrutiny, not solo claims.